Multiagent Based Reinforcement Learning (MA-RL): An Automated Designer for Complex Analog Circuits

Jiarui Bao, Jinxin Zhang, Zhangcheng Huang, *Member, IEEE*, Zhaori Bi, *Member, IEEE*, Xingwei Feng, Xuan Zeng, *Senior Member, IEEE*, and Ye Lu, *Member, IEEE*

***Abstract*—Despite the effort of analog circuit design automation, currently complex analog circuit design still requires extensive manual iterations, making it labor intensive and time-consuming. Recently, reinforcement learning (RL) algorithms have been demonstrated successfully for the analog circuit design optimization. However, a robust and highly efficient RL method to design analog circuits with complex design space has not been fully explored yet. In this work, inspired by multiagent planning theory as well as human expert design practice, we propose a multiagent-based RL (MA-RL) framework to tackle this issue. Particularly, we 1) partition the complex analog circuits into several subblocks based on topology information and effectively reduce the complexity of design search space; 2) leverage MA-RL for the circuit optimization, where each agent corresponds to a single subblock, and the interactions between agents delicately mimic the best-design tradeoffs between circuit subblocks by human experts; 3) introduce and compare *three* different multiagent RL algorithms and corresponding frameworks to demonstrate the effectiveness of the MA-RL method; 4) employing twin-delayed techniques and proximal policy to further boost training stability and accomplish higher performances; 5) the impacts of different reward function definitions as well as different state settings of MA-RL agents are investigated to further improve the robustness of this framework; and 6) experiments on *three* different complex analog circuit topologies (gain boost amplifier, delay-locked loop, and SAR ADC) and knowledge transfers between two technology nodes are demonstrated. It is shown that MA-RL framework can achieve the best-Figure of Merits for complex analog circuits' design. This work shines the light for future large scale analog circuit system design automation.**

***Index Terms*—Circuit design automation, complex analog circuits, multiagent reinforcement learning (MA-RL), proximal policy optimization (PPO), twin delayed deep deterministic policy gradient (DDPG).**

I. Introduction

DUE TO the development of Internet of Things (IoT), 5G/6G communication and edge computing, the demand for electronic chip integrated circuit (IC) design increase drastically. Digital circuit design implements functionalities based on standard cells and takes advantages of various computer aided design (CAD) tools [1]. However, the design of analog and mixed signal circuits (AMS) is still extremely labor intensive and time consuming due to their highly nonlinear behavior and complex tradeoffs among circuit Specs. It still heavily relies on the iterative optimization of analog circuit topology selection, component parameters selection, and layout routing by human design experts. It is not only time consuming but also highly designer experience dependent, making the analog circuit design an "art" with large variations.

To tackle this problem, automated design efforts for analog circuits have been increasingly investigated [2], [3], [4], [5], [6], [17]. These efforts primarily focus on three different aspects: 1) automated selection of circuit topology [14], [15]; 2) synthesis and optimization of circuit parameters, such as device sizing under a determined topology [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13]; and 3) automatic layout placement and routing [16], [17]. This work focuses on the second aspect. It is also noted some commercial EDA software begins to provide sizing functions for analog circuit, e.g., "Global optimization" tool in Cadence could achieve automated and good sizing results for some analog circuits, its underline algorithms are unrevealed to public and its capability for optimizing complex circuit with multiple subblocks is still unknown. Therefore, academic research on novel algorithms for complex analog circuit optimization is still desired, it contributes a better understanding of these algorithms for the community and it may also serve as a good supplement to existing tools in certain scenarios. Existing research works on analog circuit parameter optimization can be

Manuscript received 14 July 2023; revised 12 November 2023, 10 January 2024, and 11 April 2024; accepted 25 April 2024. Date of publication 8 May 2024; date of current version 22 November 2024. This work was supported in part by the National Key Research and Development Program of China under Grant 2020YFA0711900 and Grant 2020YFA0711901; in part by the National Natural Science Foundation of China Research under Project 62350610270, Project 62374034, Project 62235009, Project 62141407, and Project 62304052; in part by the Innovation Program of Shanghai Municipal Education Commission under Grant 2021-01-07-00-07-E00077; and in part by the Natural Science Foundation of Shanghai under Grant 22ZR1403500. This article was recommended by Associate Editor G. G. E. Gielen. *(Corresponding authors: Zhangcheng Huang; Xuan Zeng; Ye Lu.)*

Jiarui Bao, Jinxin Zhang, Xingwei Feng, and Ye Lu are with the State Key Laboratory of Integrated Chips and Systems, School of Information Science and Technology, Fudan University, Shanghai 200433, China (e-mail: lu_ye@fudan.edu.cn).

Zhangcheng Huang is with the Frontier Institute of Chip and System Shanghai, Fudan University, Shanghai 200433, China (e-mail: huangzc@fudan.edu.cn).

Zhaori Bi and Xuan Zeng are with the State Key Laboratory of Integrated Chips and Systems, School of Microelectronics, Fudan University, Shanghai 200433, China (e-mail: xzeng@fudan.edu.cn).

Digital Object Identifier 10.1109/TCAD.2024.3398554

1937-4151 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:45 UTC from IEEE Xplore. Restrictions apply.

further summarized into three categories: 1) simulation-based optimization; 2) model-based optimization; and 3) online model-based optimization. Simulation-based approach uses the SPICE simulator combined with the global algorithms as a black-box tool for parameter optimization tasks in analog circuits. Commonly used global optimization algorithms include evolutionary algorithms [3], [4], particle swarm optimization (PSO) [18], Bayesian optimization (BO) [7], [8], [9] and others. Model-based approach creates compact models to obtain fast circuit evaluation, such as optimization based on geoSpec programming [19], [20] and optimization based on neural networks, combined with global algorithms [5]. However, obtaining accurate models itself is a very challenging task as the number of circuit design parameters increases. Optimization based on online models continuously updates the model during the optimization process, combining the accuracy of simulation-based optimization with the efficiency of model-based optimization. There have been many studies on online model optimization. For example, a GSLA online algorithm combining genetic algorithm (GA) and artificial neural network (ANN) has been proposed [21], and an online algorithm based on differential evolution (DE) and a surrogate model of gaussian process (GP) has also been proposed [22]. Recently, reinforcement learning (RL) as a variant of online model optimization has been researched for analog design automation [10], [11], [12], [13]. Unlike the conventional online model-based optimization methods [21], [22], RL does not improve the agent model of circuit performance, but instead, it updates the policy model in the hope of achieving the best policy. A combination of RL and deep learning is used in [23] to optimize the size of the folding amplifier. Another combination of graph convolutional neural network (GCN) and deep deterministic policy gradient (DDPG) is used to achieve the parameter design of analog circuits and the topology transfer between different circuits [10]. It has also been shown that prioritized RL training with design knowledge could optimize analog circuits more efficiently [11].

However, the aforementioned RL methods have only been verified in relatively simple circuits, such as two-stage transimpedance amplifier [10], folded-cascode amplifier [11], and two stage operational amplifier [23]. Developing efficient and robust RL-related technologies for the design of more complex analog circuits in an automated fashion remains a challenging task. First, complex analog circuits, such as successive approximation register (SAR) ADCs and delay-locked loops (DLLs), typically contain multiple subcircuit blocks with complicated tradeoff considerations, and this greatly increases the difficulty of parameter optimization. To the best of our knowledge, the interactions between subblocks and their tradeoff relationships in these complex analog circuits have not been fully researched in the RL related framework yet. Second, complex analog circuits typically contain a large number of design parameters which may lead to search space explosion, another reason for these circuits to be under investigated particularly in RL framework.

Inspired by multiagent planning theory [24] and hierarchical design work [25], [26], we propose to partition a complex analog circuit into several subblocks based on the circuit topologies and design knowledge, and assign design goals to each subblock while also keeping in mind the overall circuit targets. This effectively reduces the complexity of the design search space and helps machine learning algorithms accomplish a more efficiently exhaustive search. Additionally, we propose to employ multiagent-based RL (MA-RL) for optimizing the circuit in an automated fashion (Fig. 1). The reward functions of MA-RL are designed for each subblock as well as the overall circuitry. Each RL agent optimizes toward its own reward, i.e., subgoal for subblock, while all agents interact with each other to accomplish the overall target. This artificial process properly mimics the design tradeoffs and iterations by human design experts. Finally, we introduce multiagent twin-delayed DDPG (MATD3) technique and multiagent proximal policy optimization (MAPPO) to further improve the training stability and achieve better results. The key contributions of this work are as follows.

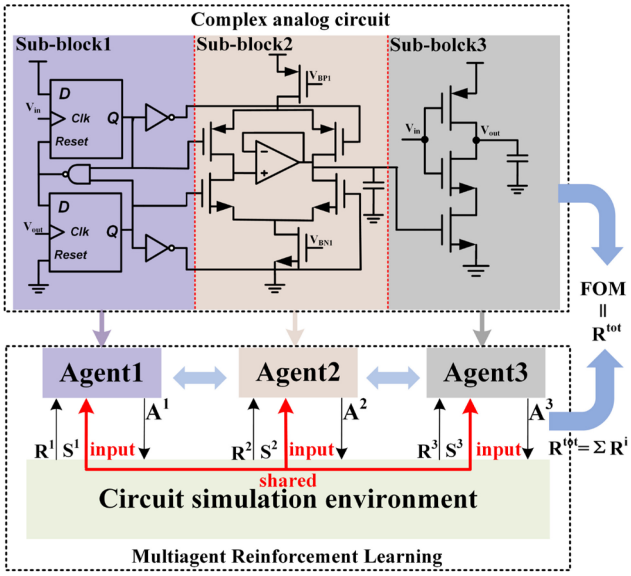


Fig. 1. Schematic of complex analog circuit partition and its design automation using multiagent RL.

1) We partition the complex analog circuits into several subblocks by leveraging circuit topology information and design knowledge, and properly create a design goal for each subblock with the consideration of overall design specifications (Specs). This effectively reduces the complexity of design search space and helps to enable a more complete search.
2) To our knowledge, we are the first to propose the MA-RL methods for analog circuit automation. Within this framework, each agent corresponds to a single subblock of the circuit, and the interactions between agents mimic the best-design tradeoffs between circuit subblocks by design experts.
3) We study the impacts of different reward definitions and state settings in MA-RL agents on performance and robustness of the algorithms, providing a more complete understanding and possible future improvement directions of the proposed framework.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:45 UTC from IEEE Xplore. Restrictions apply.

4) We investigate and compare *three* different MA-RL algorithms in this work, including MA-DDPG, the twin-delayed DDPG (TD3) algorithm which includes *double network* and *delayed* techniques, and the proximal policy optimization (PPO) which includes *clipped surrogate objective* technique to avoid training instability and accomplish better performances.
5) Finally, we demonstrate the efficiency and effectiveness of the proposed methods through *three* different circuits topologies with best-circuit performance and highest speed. We have also demonstrated the knowledge transfer between different technology nodes with a trained model.

II. Related Work

Multiagent Planning: A multiagent environment is created where a centralized planner assigns each operator to one of the agents. When a complex plan is divided into subplans with n interactive subgoals, the complexity is reduced from $O(n \times b)^d$ to $O(\max_i b_i{}^{d_i} + b \times n \times q \times d)$ due to the idea of parallelism, where n is the number of agents, b is the branching factor, d is the depth of the problem and q is a measure of the positive interactions between overlapping propositions, $b_i \approx b/n$ and $d_i \approx d/n$ [24].

MA-RL: RL is a machine learning technique to employ intelligent agents to take actions in an environment in order to maximize the cumulative reward. It has been proposed to undertake complex intelligent tasks, such as playing games [27] and controlling robotics [28]. MA-RL adapts actor–critic methods that consider action policies of other agents and is able to successfully learn policies that require complex multiagent coordination. This approach shows strength compared to existing methods in cooperative as well as competitive scenarios [29]. Here, we propose to use MA-RL to optimize complex analog circuits.

DDPG: DDPG is the deterministic policy gradient algorithm for RL with continuous actions. The deterministic policy gradient is the expected gradient of the action-value function, and it can be estimated much more efficiently than the usual stochastic policy gradient. Its deterministic actor–critic can significantly outperform its stochastic counterparts in high-dimensional action spaces [30].

TD3: TD3 uses three modifications to reduce value function overestimation [31]. First, it learns two Q-value functions and uses the minimum value function estimate during policy updates. Second, TD3 agent updates the policy and targets less frequently than the Q functions. Third, a TD3 agent adds noise to the target action when updating the policy. Here we introduce TD3 techniques in the MA-RL framework.

PPO: PPO is a policy-based RL algorithm, designed to effectively use limited data while maintaining the stability of learning performance [32]. The techniques of *clipping ratio* and *clipped objective function* are applied in PPO to limit the amplitude of policy updates, ensuring proximity between the new and old policies. It helps to prevent training instability caused by excessive policy updates.

III. Methodology

A. Problem Formulation

The analog circuit design optimization can be formulated as a bound-constrained optimization

$$\underset{x \in D^n}{\text{maximize}} \ \text{FoM}(x) \tag{1}$$

where x is the parameter vector, n is the number of parameters to search, and D^n is the design space. Figure of Merits (FoM) is the objective we aim to optimize, and it is the sum of FoM for each subblock

$$\text{FoM}(x) = \sum_{i \in N} \text{FoM}^i\left(x^i\right) = \sum_{i \in N} R^i \tag{2}$$

where x^i is the parameter vector in the ith subblock and N is the number of all subblocks. $\text{FoM}^i(x^i)$ is the objective we aim to optimize in the ith subblock, and it is equivalent to the reward R^i for this subblock in the multiagent RL implementation. The reward R^i includes the reward R_i calculated from the performance Specs of the ith subblock and the reward R_t calculated from the performance Specs of the entire circuit, and the detailed definition is shown in (7)–(13). This reward definition takes into account the contribution of x^i to both the performance of the ith subblock and that of the entire circuit.

B. Framework Overview

An overview of the proposed framework is shown in Fig. 1.

1) The complex analog circuit is partitioned into several subblocks based on the topology information and design knowledge. Conventionally a complex circuit can be viewed as a combination of electrical components and circuit nodes, where the currents of the components and the voltages of the nodes obey Kirchhoff's law. Specially, there are some nodes who can form a boundary where the currents and voltages within or on the nodes are relatively independent of outside components, and this naturally divides the complex circuit into different subblocks, one example is explained in [33].
2) A MA-RL framework is adopted for the design automation task, and each subblock is assigned to a corresponding RL agent.
3) The individual RL agent trains a model to optimize its reward, i.e., the FoM^i of the subblock i, by interacting with all the agents through circuit simulation environment.
4) The overall FoM is eventually achieved by the cooperation and competition between all subblock agents during the algorithm execution process.

C. Multiagent Reinforcement Learning Formulation

As shown in Fig. 2, the *key* concept of the MA-RL framework is centralized training with decentralized execution [29]. Multiple agents are created where actor–critic RL agents are used. The actor is the policy structure searching for the action to optimize the FoM, while the critic informs the actor how good the action is and how it should improve. Critics in different agents interact with each other by shared

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:45 UTC from IEEE Xplore. Restrictions apply.

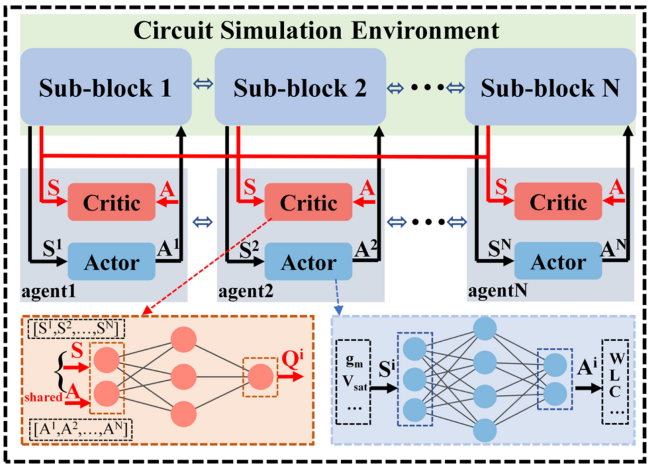


Fig. 2. Detailed diagram of multiagent actor–critic RL implementation on circuit design automation.

states and actions of all agents, and then they influence their corresponding actor network by changing their loss function.

Actor and Critic: In the implementation of MA-RL framework, both Actor and Critic are ANNs with one input layer, one output layer, and two hidden layers. The input of the actor network is its *state* and the output is the *action* as described below. The input of the critic network is a concatenation of all states and actions, and its output is Q value [29].

$$\text{Input of the critic network} = \left[\text{State}^1, \text{State}^2, \ldots \text{State}^N, \text{Action}^1, \text{Action}^2, \ldots, \text{Action}^N\right]. \tag{3}$$

Due to the fact that each critic network in MA-RL can access information from all agents, i.e., each critic network's input is a concatenation of the states and actions of all agents, the optimization process of each subblock indeed takes into account the impacts of all subblocks.

Action Space: An action is the output of the actor network in one particular agent, and it is composed of design parameters, such as devices' sizes and passives, e.g., transistor width (W), length (L), capacitance (C), resistance (R), etc. Practically, it is a vector concatenating this information of its tunable devices within its corresponding subblock

$$\text{Action}^i = \left[W_1^i, L_1^i, W_2^i, L_2^i, \ldots, W_k^i, L_k^i, R_1^i, R_2^i, \ldots, C_1^i, C_2^i \cdots\right]. \tag{4}$$

Note all values of these design parameters are scaled and normalized in the range between -1 and 1 for the network training purpose.

State Space: In the MA-RL, the state of each agent is a vector containing the circuit information of the corresponding subblock observed at the moment. Two state vector definitions are tested and discussed in this work.

The first definition involves a set of simulated operating conditions for a subblock, such as g_m, V_{sat}, etc., taking the state of agent i as an example

$$\text{State}^{i,a} = \left[\text{gm}_1^i, \text{gm}_2^i, \ldots, V_{\text{sat}}^i, V_{\text{sat}}^i, \ldots\right]. \tag{5}$$

In the second case, the state of each agent is a vector composed of the current performance reached in the corresponding subblock, and can be defined as

$$\text{State}^{i,b} = \left[\text{Spec}_1^i, \text{Spec}_2^i, \ldots, \text{Spec}_n^i\right] \tag{6}$$

where Spec_n^i is the nth performance Spec of the ith subblock. For example, the state of each subblock is a set of circuit performance, such as its gain, power consumption, and so on. The comparisons of these two state definitions are tested and discussed in Section IV.

Reward: The reward R^i provides optimization goal for agent i. It takes into account of current action's impact on both the performance of the ith subblock and that of the entire circuit, and it is defined as the sum of these two parts

$$R^i = R_i + R_t. \tag{7}$$

R_i is defined as the weighted sum of the rewards obtained by the performance Specs of the ith subblock, and calculated as

$$R_i = \sum_{l \in M^i} k_l^i r_l^i \tag{8}$$

where M^i is the number of Specs of the ith subblock, k_l^i is the weight of the lth performance Spec s_l^i, and the reward of the lth performance Spec r_l^i is defined as

$$r_l^i = \begin{cases} -1, & \text{if } s_l^i > s_l^{i,\max} \text{ and } s_l^i \text{ is minimized} \\ -1, & \text{if } s_l^i < s_l^{i,\min} \text{ and } s_l^i \text{ is maximized} \\ \dfrac{s_l^i - s_l^{i,\min}}{s_l^{i,\max} - s_l^{i,\min}}, & \text{if } s_l^i > s_l^{i,\min} \text{ and } s_l^i \text{ is maximized} \\ \dfrac{s_l^{i,\max} - s_l^i}{s_l^{i,\max} - s_l^{i,\min}}, & \text{if } s_l^i < s_l^{i,\max} \text{ and } s_l^i \text{ is minimized} \end{cases} \tag{9}$$

where s_l^i is the lth Spec target in the ith subblock, $s_l^{i,\min}$, and $s_l^{i,\max}$ are predefined boundary value of s_l^i. For those subblock Specs which are not included in the entire circuit design Specs, the Spec boundaries are used for the reward normalization.

R_t is calculated based on the Specs of the entire circuit, and it is defined as the weighted sum of the rewards obtained by M performance Specs

$$R_t = \sum_{j \in M} k_j^t r_j^t \tag{10}$$

where M is the total number of Specs, k_j^t is the weight of the jth performance Spec s_j^t, and r_j^t is the reward of the jth performance Spec. Two definitions of r_j^t, denoted as $r_j^{t,a}$ and $r_j^{t,b}$ are also tested and discussed in Section IV.

$r_j^{t,a}$ is defined as

$$r_j^{t,a} = \begin{cases} -1, & \text{if } s_j^t > s_j^{t,\max} \text{ and } s_j^t \text{ is minimized} \\ -1, & \text{if } s_j^t < s_j^{t,\min} \text{ and } s_j^t \text{ is maximized} \\ \dfrac{s_j^t - s_j^{t,\min}}{s_j^{t,\max} - s_j^{t,\min}}, & \text{if } s_j^t > s_j^{t,\min} \text{and } s_j^t \text{ is maximized} \\ \dfrac{s_j^{t,\max} - s_j^t}{s_j^{t,\max} - s_j^{t,\min}}, & \text{if } s_j^t < s_j^{t,\max} \text{and } s_j^t \text{ is minimized} \end{cases} \tag{11}$$

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:45 UTC from IEEE Xplore. Restrictions apply.

where $s_j^{t,\min}$ is defined as the target value of a Spec if that particular Spec is the smaller the better, while $s_j^{t,\max}$ is defined as the target value of a Spec if that particular Spec is the larger the better, otherwise they are set to be the worst boundary. Taking gain as an example, if gain of a circuit is expected to be 10 dB or greater, then $s_j^{t,\max}$ is set to be 10 and $s_j^{t,\min}$ is set to be 0. If s_j^t is worse than the worst-boundary value, -1 is assigned as the score for this reward.

$r_j^{t,b}$ is defined referring to [34]. Particularly, a clipping mechanism is introduced for Spec reward definition to alleviate the problem of the over-shooting of certain Specs while underweighting others. The clipping reward of the jth performance Spec $r_j^{t,b}$ can be divided into $r_{j,u}^t$ and $r_{j,d}^t$, each defined as the following:

$$r_{j,u}^{t,b} = \begin{cases} -1, & \text{if } s_{j,u}^t < s_{j,u}^{t,\min} \\ \min\left(1, \frac{s_u^t - s_{j,u}^{t,\min}}{s_{j,u}^{t,\max} - s_{j,u}^{t,\min}}\right), & \text{if } s_{j,u}^t > s_{j,u}^{t,\min} \text{ and } \sum_{j\in M} r_j^t < M \\ \frac{s_u^t - s_{j,u}^{t,\min}}{s_{j,u}^{t,\max} - s_{j,u}^{t,\min}}, & \text{if } s_{j,u}^t > s_{j,u}^{t,\min} \text{ and } \sum_{j\in M} r_j^t \geq M \end{cases} \tag{12}$$

$$r_{j,d}^{t,b} = \begin{cases} -1, & \text{if } s_{j,d}^t > s_d^{t,\max} \\ \min\left(1, \frac{s_{j,d}^{t,\max} - s_{j,d}^t}{s_{j,d}^{t,\max} - s_{j,d}^{t,\min}}\right), & \text{if } s_{j,d}^t < s_{j,d}^{t,\max} \text{ and } \sum_{j\in M} r_j^t < M \\ \frac{s_{j,d}^{t,\max} - s_{j,d}^t}{s_{j,d}^{t,\max} - s_{j,d}^{t,\min}}, & \text{if } s_{j,d}^t < s_{j,d}^{t,\max} \text{ and } \sum_{j\in M} r_j^t \geq M \end{cases} \tag{13}$$

where $r_{j,u}^t$ represents the reward for the Spec $s_{j,u}^t$, which is the larger the better. $r_{j,d}^t$ represents the reward for the Spec $s_{j,d}^t$, which is the smaller the better. $s_{j,u}^{t,\max}$and $s_{j,d}^{t,\min}$ are defined as the target values of the Spec $s_{j,u}^t$ and the Spec $s_{j,d}^t$, respectively, while $s_{j,u}^{t,\min}$and $s_{j,d}^{t,\max}$ are defined as the worst-boundary values. When a Spec fails to meet the worst boundary, the reward of the Spec is set to -1. When a Spec meets the design objective, but the entire circuit does not satisfy its overall design objective, the reward is fixed at 1. Once the entire circuit meets its overall design objective, the reward of this particular Spec continues to increase and it is further optimized.

D. Algorithm Execution Flow

In the optimization process, we first leverage multiagent DDPG (MADDPG) algorithm [29], and the detailed algorithm flow is shown in Algorithm 1. x= $(S^1 \ldots, S^N)$ and $x'(S^{1'}, \ldots, S^{N'})$ denote the current and the next set of states of the circuit with N subblocks. Similarly, $A(A^1, \ldots, A^N)$ and $R(R^1, \ldots, R^N)$ denote the set of actions and rewards of N subblocks, respectively. Each agent in MADDPG contains a critic network and an actor network. The critic network provides a Q value which contains the reward and future discounted reward of subblock. The actor network is trained to get an action that maximizes the Q value.

Noted that DDPG requires a stable environment for training, while subblocks of a circuit interact with each other, therefore these subblocks cannot be trained by multiple parallel non-interactive DDPG algorithms. MADDPG takes advantage of centralized training with decentralized execution to solve this problem.

Algorithm 1: MADDPGfor Circuit Optimization

Create N agents for complex analog circuit's N sub-blocks
Randomly ihitialize the actor network $\mu_i\left(S^i \mid \theta_i^\mu\right)$ and the critic network $Q_i\left(S^i, A^i \mid \theta_i^Q\right)$ in the i-th agent for all $i \in N$;
Initialize replay buffer D and batch size B;
for $t = 1$ *to* T **do**
  Randomly generate initial states in each agent $x = \left(S^1, \ldots, S^N\right)$;
  if [1] size $(D) < B$ **then**
    Generate random sample action $A^i \leftarrow \mu_i\left(S^i \mid \theta_i^\mu\right)$ in the i-th agent for $i \in N$
  else
    Select action for the i-th agent according to policy and exploration noise $\sigma, A^i \leftarrow \mu_i\left(S^i \mid \theta_i^\mu\right) + \sigma$;
    end if[1]
  Use $A\left(A^1, \ldots, A^N\right)$ as input for circuit simulation and observe new state $x'\left(S^{1\prime}, \ldots, S^{N\prime}\right)$ and reward $R\left(R^1, \ldots, R^N\right)$;
  Store transition $\left(x, A, R, x'\right)$ in replay buffer D;
  if [2] size $(D) > B$ **then**
    Sample a batch of B transitions $\left(\hat{x}, \hat{A}, \hat{R}, \hat{x}'\right)$ in D,
    for agent $i = 1$ *to* N **do**
      $\hat{A}^{i'} \leftarrow \mu\left(\widehat{S^i} \mid \theta_i^\mu\right)$;
      $y_i \leftarrow r_i + \gamma \cdot Q_i'\left(\hat{x}', \hat{A}^{1'}, \ldots, \hat{A}^{N'}\right)$;
      update the critic network in the i-th agent by minimizing loss function: $L = \frac{1}{B}\sum\left(Q_i\left(\hat{x}, \hat{A}^1, \ldots, \hat{A}^N\right) - y_i\right)^2$;
      update actor network in i-th agent by the gradient ;
      $\nabla_{\theta_i} J\left(\theta_i^\mu\right) =$
      $\frac{1}{B}\sum \nabla_{a_i} Q_i\left(\hat{x}, \hat{A}^1, \ldots, \hat{A}^N\right) \nabla_{\theta_i^\mu} \mu_i\left(S^i \mid \theta_i^\mu\right)\Big|_{\hat{S}^i}$;
    end for
    end if[2]
end for

1) In the execution phase, MADDPG establishes a replay buffer D to store the sampled transitions (x, A, R, x'). If the number of transitions in D is less than the batch size B, the untrained actor networks are used to generate random design parameters, which are then used for circuit simulation. However, once the number of transitions in D becomes larger than the batch size B, predictions of the optimal policy are made using the actor networks that had been trained. In the execution phase, a particular actor network only needs the information of its own local agent (S^i in x) to generate an action A^i. At the same time, exploration noise σ is added to the action A^i to help reach the global optimal point. The action vector $A(A^1, \ldots, A^N)$ is provided to the circuit for circuit simulation, and then the next state x' and reward R for all subblock circuits can be observed.
2) The centralized training phase is based on the transitions (x, A, R, x') collected in the execution phase. Concretely, each agent takes the information from all agents to train its critic network, and then the output of the critic network of an agent influences its corresponding actor network by changing its loss function. For example, in a complex analog circuit with N subblocks, the performance of the ith subblock is influenced not only by its own design parameters but also by those in other subblocks. In the execution in MADDPG framework,

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:45 UTC from IEEE Xplore. Restrictions apply.

each agent is able to take into account the design parameters and state vectors from all subblocks for calculating the Q value both for the current state and that of the next state. Combining the reward R^i and Q', the state-action value function $y_i$of the ith agent is obtained using the Bellman equation, as described in Algorithm 1. The critic network is updated based on the loss function of the current Q value and y_i. Subsequently, the estimated Q value obtained by the critic network is used to compute policy gradients, which are then used to update the actor network. The centralized training method of MADDPG enables each agent to learn and interact effectively in an environment affected by the influences of other subblocks and is suitable for complex analog circuits that require multiple subblocks for tradeoff optimization.

3) For MA-RL algorithm termination, a maximum number of simulation times is first set based on the design complexity of a particular circuit, e.g., 15 000 for a gain boost amplifier (GBA) circuit case. A particular design solution is updated and recorded if its reward is higher than the previous one. Finally, if the design solution does not reach the design Specs for the entire 15 000 runs, the one with the highest reward will be output. On the other side, if the design Specs are reached for a particular run and the result of this run is not updated for another 1000 simulation runs, we consider the result is converged and the algorithm will be terminated.

It is noted prior art a "Done" flag is incorporated in the termination condition [35], this has also been studied for the complex analog circuit design optimization task in Section IV. The "Done" flag is a Boolean variable assigned either 0 or 1, where the change in assignment indicates the end of an episode and the start of a new one. When the "Done" flag is introduced, the Bellman equation for each agent in Algorithm 1 is modified as follows:

$$y_i \leftarrow r_i + \gamma . Q_i'\left(\widehat{x}', \widehat{A}^{1'}, \ldots, \widehat{A}^{N'}\right)(1 - \text{Done}). \tag{14}$$

E. Enhancing RL Agent With Twin Delayed Technique

Similar to DQN's max operation on the value function, MADDPG also tends to overestimate the value function [31]. In circuit automation task, it may preferentially select circuit transitions (A, S, S', R) with a large value that is overestimated and finish circuit design in a suboptimal point. Furthermore, DDPG algorithm appears to suffer from weak robustness and stability.

To further improve design automation quality and efficiency. Multiagent twin delayed DDPG (TD3) algorithm is introduced for model training [31], and there are three major techniques added in TD3.

1) Double Network: Two critic networks instead of one are employed for value function evaluation, and the lower-value one is used for the target update

$$y_i \leftarrow r_i + \gamma . \min\left(Q'_{1,i}\left(\widehat{x}', \widehat{A}^{1'}, \ldots, \widehat{A}^{N'}\right) Q'_{2,i}\left(\widehat{x}', \widehat{A}^{1'}, \ldots, \widehat{A}^{N'}\right)\right). \tag{15}$$

Algorithm 2: MATD3 for circuit optimization.

Provide N agents for complex analog circuit's N sub-blocks Randomly initialize the actor network $\mu_i(S^i \mid \theta_i^{\mu})$ and two critic networks $Q_{1,i}\left(S^i, A^i \mid \theta_i^{Q_1}\right), Q_{2,i}\left(S^i, A^i \mid \theta_i^{Q_2}\right)$ in the i-th agent;
The pseudo-code in the next part is the same as that in Algorithm 1 except if [2] loop is replaced by if [3] loop as shown below:
if [3] size $(D) > B$ **then**
  Sample a batch of B transitions $\left(\hat{x}, \hat{A}, \hat{R}, \hat{x}'\right)$ in D;
  for agent $i = 1$ *to* N **do**
    Obtain target actor according to the actor network and regularization noise $\epsilon, \hat{A}^{i'} \leftarrow \mu\left(\widehat{S}^i \mid \theta_i^{\mu}\right) + \epsilon$; $y_i \leftarrow r_i + \gamma \cdot \min\left(Q'_{1,i}\left(\hat{x}', \hat{A}^{1'}, \ldots, \hat{A}^{N\prime}\right), Q'_{2,i}\left(\hat{x}', \hat{A}^{1'}, \ldots, \hat{A}^{N\prime}\right)\right)$;
    update the critic networks in the i-th agent by minimizing loss function: $L\left(\theta_i^{Q_1}\right) \leftarrow \frac{1}{B}\sum\left(Q_{1,i}\left(\hat{x}, \hat{A}^1, \ldots, \hat{A}^N\right) - y_i\right)^2$;
    $L\left(\theta_i^{Q_2}\right) \leftarrow \frac{1}{B}\sum\left(Q_{2,i}\left(\hat{x}, \hat{A}^1, \ldots, \hat{A}^N\right) - y_i\right)^2$;
    if [4] t mod d **then**
      update actor network in i-th agent by the gradient ; $\nabla_{\theta_i} J(\theta_i^{\mu}) = \frac{1}{B}\sum \nabla_{a_i} Q_{1,i}\left(\hat{x}, \hat{A}^1, \ldots, \hat{A}^N\right) \nabla_{\theta_i^{\mu}} \mu_i(S^i \mid \theta_i^{\mu})\Big|_{\hat{S}^i}$;
      end if[4]
  end for
  end if[3]

This mitigates the risk of value function overestimation caused by training noise. Similar to that of Algorithm 1, the Bellman equation for each agent in Algorithm 2 needs to be modified as the following when the "Done" flag is introduced:

$$y_i \leftarrow r_i + \gamma . \min\left(Q'_{1,i}\left(\widehat{x}', \widehat{A}^{1'}, \ldots, \widehat{A}^{N'}\right) Q'_{2,i}\left(\widehat{x}', \widehat{A}^{1'}, \ldots, \widehat{A}^{N'}\right)\right)(1 - \text{Done}). \tag{16}$$

2) Delayed Policy Update: Actor network is updated at a lower frequency than critic networks to minimize the expectation of TD-error and improve the stability of circuit strategical training.

3) Target Policy Smoothing Regularization: Normal distributed noise is added to the training actions in critical network for regularization, and this is to reduce ANN overfitting and improve the stability of circuit training. The execution flow of these changes is detailed in Algorithm 2.

As an initial test, we compare the robustness of the TD3 and DDPG algorithms in the optimization of a rail-to-rail OTA [Fig. 3(a)] designed on commercial 0.13-μm mixed-signal CMOS process PDK. The rail-to-rail OTA has 36 tunable design parameters and 7 design objectives. The design objectives are defined as follows:

$$\begin{aligned} &\text{Open Loop Gain} \geq 100 \text{ dB} \\ &\text{Unity Gain Frequency} \geq 50 \text{ MHz} \\ &\text{Power} \leq 200\mu \text{ W} \\ &\text{Slew Rate} \geq 10\text{V}\mu s \\ &\text{PSRR} \geq 80 \text{ dB} \\ &\text{CMRR} \geq 100 \text{ dB} \\ &\text{Input common} - \text{mode range} = 1.2\text{V}. \end{aligned} \tag{17}$$

In the comparison, the rewards are calculated based on (10) and (11), and then normalized according to the design Specs.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:45 UTC from IEEE Xplore. Restrictions apply.

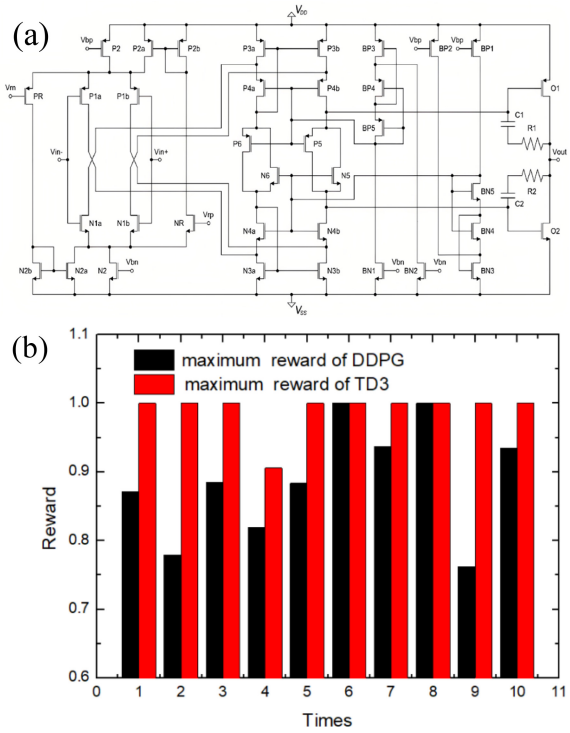


Fig. 3. (a) Schematic of rail-to-rail OTA. (b) Comparison of robustness between TD3 and DDPG algorithms in rail-to-rail OTA.

For design points that meet or exceed the design Spec targets in all performances, their rewards are set to 1. Fig. 3(b) shows DDPG successfully achieves the optimization task in two out of ten rounds while TD3 always accomplishes the final goal. We have also repeated these tests and we found in general DDPG's success rate is between 20%–40% in the case we have tested, while TD3 exhibits better robustness.

F. Enhancing RL Agent With Proximal Policy Optimization

The MAPPO algorithm is also investigated in this work. Unlike value-based MADDPG, MAPPO is believed to be immune from the problem of over-estimation of value function caused by maximizing bias. In PPO, the output of the actor network is not a deterministic action but rather the mean and variance of an action distribution [32]. The actor network, with the assistance of the critic network, updates its parameters by maximizing a surrogate objective function. The objective is to find actions that can maximize the expected rewards and increase their corresponding probabilities.

To prevent training instability due to over-frequent updating of network parameters in the policy optimization, a technique called *Clipped Surrogate Objective* is applied in PPO. It introduces a clipping function to safe guide the gap between the new and old policies, and it uses a parameter ε to control the degree of clipping. This helps enhance the stability of the algorithm and leads to more optimal solutions. In this work, PPO is used in the MA-RL framework and the detailed flow of MAPPO is described in Algorithm 3.

Algorithm 3: MAPPO for Circuit Optimization

Create N agents for complex analog circuit's N sub-blocks Randomly initialize the actor network $\mu_i\left(S^i \mid \theta_i^\mu\right)$ and the critic network $Q_i\left(S^i, A^i \mid \theta_i^Q\right)$ in the i-th agent for all $i \in N$; Initialize replay buffer D and batch size B and ppo epoch M;
for $t = 1$ *to* T **do**
  Randomly generate initial states in each agent $x = \left(S^1, \ldots, S^N\right)$;
  if [5] size $(D) < B$ **then**
    Sampling action for the i-th agent according to policy $A^i \leftarrow P^i \leftarrow \mu_i\left(S^i \mid \theta_i^\mu\right)$;
    Use $A\left(A^1, \ldots, A^N\right)$ as input for circuit simulation and observe new state $x'\left(S^1, \ldots, S^{N\prime}\right)$ and reward $R\left(R^1, \ldots, R^N\right)$;
    Store transition $\left(x, A, R, x'\right)$ in replay buffer D;
    end if[5]
  if [6] size$(D) > B$ **then**
    Sample a batch of B transitions $\left(\hat{x}, \hat{A}, \hat{R}, \hat{x}'\right)$ in D;
    for agent $i = 1$ *to* N **do**
      $\hat{A}^{i'} \leftarrow \hat{P}^{i'} \leftarrow \mu\left(\hat{S}^i \mid \theta_i^\mu\right)$;
      $y_i \leftarrow r_i + \gamma \cdot Q_i'\left(\hat{x}', \hat{A}^{1'}, \ldots, \hat{A}^{N'}\right)$;
      compute advantage in the i-th agent: $A_{adv}^i \leftarrow y_i - Q_i'\left(\hat{x}', \hat{A}^{1'}, \ldots, \hat{A}^{N'}\right)$;
      for $m = 1$ *to* M **do**
        update critic network in the i-th agent by maximizing the objective function: $L = \frac{1}{B}\sum\left(Q_i\left(\hat{x}, \hat{A}^1, \ldots, \hat{A}^N\right) - y_i\right)^2$;
        update actor network in the i-th agent by maximizing the objective function: $L(\theta_i^\mu) = \widehat{E}\left[\min\left(\frac{\mu_i(s^i|\theta_i^\mu)}{\mu_i^{\text{old}}(s^i|\theta_i^\mu)}\hat{A}_{adv}^i, \text{clip}\left(\frac{\mu_i(s^i|\theta_i^\mu)}{\mu_i^{\text{old}}(s^i|\theta_i^\mu)}, 1-\varepsilon, 1+\varepsilon\right)\hat{A}_{adv}^i\right)\right]$;
      end for
    end for
    Empty the replay buffer D;
    end if[6]
end for

IV. Experiments

To demonstrate the effectiveness of the proposed MA-RL methodology, we experiment with the reference methods and our proposed MA-RL (MADDPG, MATD3, and MAPPO) to three real-world circuits' design on different technology nodes, a GBA designed on commercial 0.18-μm mixed-signal CMOS process PDK, a DLL circuit on 0.13-μm mixed-signal CMOS process PDK, an SAR ADC on 65-nm mixed-signal CMOS process PDK. More specifically, the device sizing tasks for all circuits are accomplished, where transistor W, L and passive device are optimized. The circuit simulator is Synopsys HSpice. The experiments are run on Intel Core i9-10980XE CPU with a base frequency of 3.0 GHz. We compared our proposed MA-RL methods MADDPG, MATD3, and MAPPO with human design, simple Monte Carlo (MC), GA referenced in [5], BO reference in [10], and single agent RL techniques. In terms of the single agent RL method, we experimented the standard vanilla RL with DDPG agent used in [10], [12], and [13]. It is noted that all algorithms are tested with the same number of simulations for a fair comparison.

A. Circuits and Their MA-RL Formulation for the Experiments

1) Gain Boost Amplifier: To illustrate the workflow and effectiveness of MA-RL, a fully differential GBA [Fig. 4(a)] is first designed in this work. The main structure of GBA

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:45 UTC from IEEE Xplore. Restrictions apply.

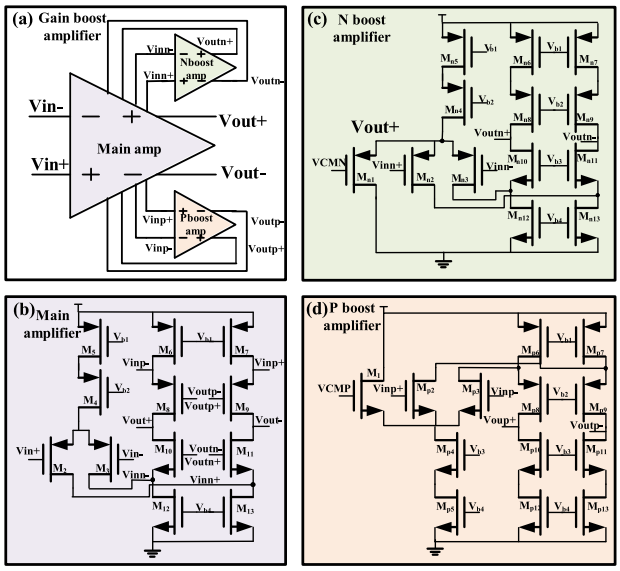


Fig. 4. Schematic of (a) gain boost amplifier circuits, (b) main amplifier, (c) N boost amplifier, and (d) P boost amplifier.

is a fully differential folded-cascode [Fig. 4(b)]. To further improve the gain, two assistant amplifiers with the fully differential folded-cascode topology are utilized between the source and gate terminals of the common-gate transistors, as shown in Fig. 4(c) and (d), respectively. The specification targets of GBA include (a) gain, (b) unity gain frequency (UGF), (c) power consumption, (d) phase margin (PM), and (e) slew rate (SR). The design objectives are defined as follows:

$$\begin{aligned} &\text{DC Gain} \geq 105 \text{ dB} \\ &\text{Unity Gain Frequency} \geq 10 \text{ MHz} \\ &80 \text{ deg} \leq \text{Phase Margin} \leq 90 \text{ deg} \\ &\text{Power} \leq 5 \text{ mW} \\ &\text{Slew Rate} \geq 10 \text{ V}\mu\text{s}. \end{aligned} \tag{18}$$

In the GBA circuit, a total of 21 independent design parameters are selected as optimization variables, including seven widths of transistors in main amplifier (W_2, W_4, W_5, W_6, W_8, W_{10}, and W_{12}), seven widths of transistors in N boost amplifier (W_{n1}, W_{n2}, W_{n4}, W_{n5}, W_{n6}, W_{n8}, and W_{n10}), and seven widths of transistors in P boost amplifier (W_{p1}, W_{p2}, W_{p4}, W_{p5}, W_{p6}, W_{p8}, and W_{p10}). Other design parameters are treated as dependent parameters obtained from the independent design parameters. In the case of the main amplifier, the widths of W_3, W_7, W_9, W_{11}, and W_{13} are equal to the widths of W_2, W_6, W_8, W_{10}, and W_{12}, respectively, due to the symmetry of the difference circuit. The search range for these design parameters is set to [5, 105] μm.

The source and gate of the common-gate transistors in the main amplifier are special nodes, and two assistant amplifiers between the nodes form individual circuit subblocks. The MOS transistors within each assistant amplifier are bound to each other, forming a relatively closed design space. Therefore, the GBA is naturally partitioned into three subblocks for the MA-RL formulation. Thus, a three agents MA-RL framework is employed here. The three agents are denoted as Agent^1, Agent^2, and Agent^3, each assigned to an amplifier in Fig. 4(b)–(d), respectively.

In the MA-RL framework of this example, the action vectors of the three agents are the lists of design parameters corresponding to their respective circuit subblocks, denoted as Action^1, Action^2, and Action^3. The state vectors of the three agents are the lists of overdrive voltages (V_{od}) of the MOS transistors to be optimized in their respective circuit subblocks, denoted as State^1, State^2, and State^3. The rewards of the three agents are denoted as R^1, R^2, and R^3, respectively. And according to (7), the reward for each agent is defined as the sum of two parts. For example, the reward for Agent^1 is the sum of the entire circuit reward R_t and the reward for the main amplifier R_{main}.

The reward $R_t$is obtained by normalizing and then weighted summation of Specs (a)–(e), as defined in (10). The boundary for gain of GBA is set between [5, 105] dB; the boundary for UGF of GBA is set between [0, 10] MHz; the boundary for PM of GBA is set between [0, 85] deg, and for PM greater than 90 degrees, $r^t_{\text{PM}}$is set to -1; the boundary for power of GBA is set between [5], [10] mW, and r^t_{power} is set to -1 with power larger than 10 mW; the boundary for SR is set between [0, 10] V/μs. After evaluating the difficulty of optimizing each Spec, the weights for gain, UGF, PM, power, and SR in (10) are set as 1/8, 3/8, 1/8, 3/4, and 1/8, respectively. For each circuit subblock, the corresponding reward is calculated based on the gain, UGF, and power of the subblock amplifier itself, according to (8) and (9). In this example, the setting of weights and boundaries of the Specs in the subblocks are consistent with that used in the reward R_t. During the training process of MA-RL, each subblock optimizes its performance through tradeoffs with other subblocks and specific design objectives are not needed.

In the MA-RL framework, the learning and execution principles of each agent are similar to the common RL algorithm. Importantly, MA-RL employs a method of centralized training with distributed execution. Taking the main amplifier as an example, the actor network of Agent^1 only needs the state of the main amplifier (State^1) to output the corresponding action (Action^1), while Agent^1 is able to access the information of the entire circuit through the critic network, for which the input is a concatenation of the states and actions of all three agents

$$\left[\text{State}^1, \text{State}^2, \text{State}^3, \text{Action}^1, \text{Action}^2, \text{Action}^3\right]. \tag{19}$$

This allows Agent^1 to take into account the influence of all other agents during the learning process, performing tradeoff optimization among subblocks to optimize the performance of the entire circuit.

2) Delay Locked Loop: A more complex DLL with 42 tunable design parameters, including transistor sizes and passives, is also designed to demonstrate the capability of MA-RL framework. DLL can be considered as a feedback circuit that phase lock an output to an input without the use of an oscillator, and its schematic and timing diagram are shown in Fig. 5(a) and (b). Based on its circuit topology, it can be partitioned naturally to three subblocks, i.e., phase detector

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:45 UTC from IEEE Xplore. Restrictions apply.

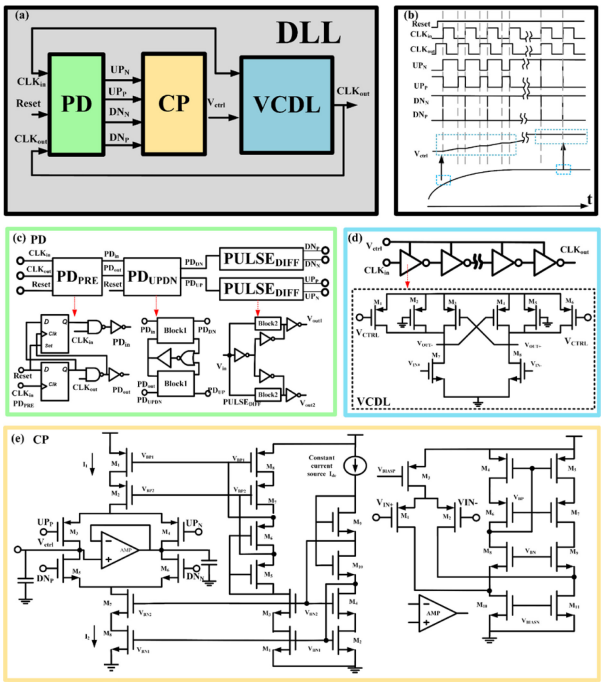


Fig. 5. Schematic of (a) DLL circuits, (b) timing diagram, (c) PD block, (d) VCDL block, and (e) CP block.

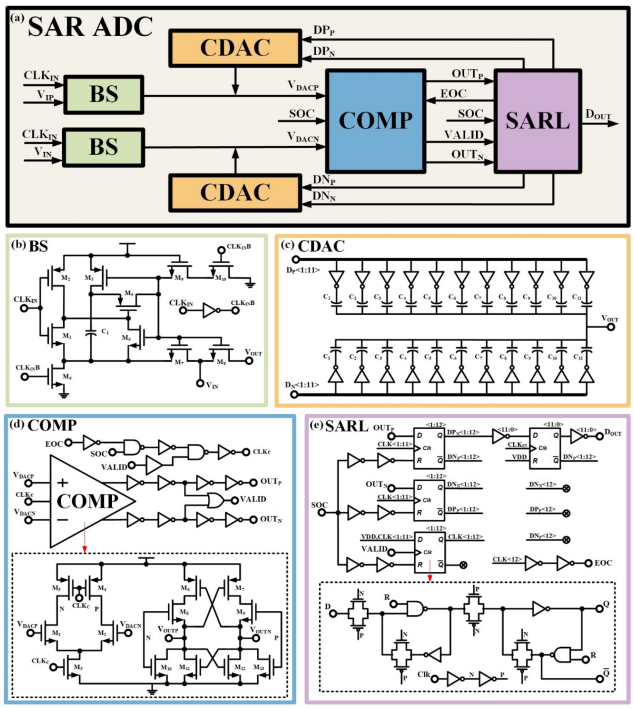


Fig. 6. Schematic of (a) SAR ADC circuits, (b) BS block (c) CDAC, (d) comparator block, and (e) successive approximation register logic block.

(PD), charge pump (CP) and voltage-controlled delay line (VCDL). PD is mainly composed of standard cells and it is used to detect phase difference between reference clock and output clock feedbacked from VCDL. There is no device sizing task for PD and therefore its design is fixed. The schematic of VCDL is shown in Fig. 5(d). Phase difference approaches to zero and V_{ctrl} becomes fixed once delay time of VCDL is equal to the period of the reference clock. CP contains a charge bump, bias circuit and buffer amplifier as shown in Fig. 5(e). The typical 0.1 ns, period of output clock can be regarded as identical to the reference clock.

A two agents RL framework is employed for this exercise, one assigned to CP and the other assigned to VCDL. The overall specifications of DLL include (a) working frequency range, (b) lock time (t_{lock}) of circuit, (c) power consumption, (d) difference of phase, and (e) jitter. The design objectives are defined as follows:

$$\begin{aligned} &\text{Phase Difference} \leq 100 \text{ ps} \\ &\text{Lock Time} \leq 1\mu s \\ &150 \text{ MHz} \leq \text{Working Frequency Range} \leq 250 \text{ MHz} \\ &\text{Power} \leq 25\text{mW} \\ &\text{Jitter} \leq 300 \text{ ps}. \end{aligned} \tag{20}$$

Among those, (a)–(e) are normalized and added up as reward R_t for this circuit as defined in (10). For agent 1 corresponding to CP, the difference of I_1 and I_2, V_{ctrl} and lock time, gain of the amplifier within CP are normalized using (8) as r_1^{CP}, r_2^{CP}, r_3^{CP}, r_4^{CP}, respectively, and weighted sum as reward R^{CP}. The reward for agent 1 (R^1) is the sum of R_t and R^{CP}. For agent 2 corresponding to VCDL, frequency range is used as reward R^{VCDL}, and the reward for agent 2 (R^2) is the sum of R_t and R^{VCDL}. The overall reward R_{tot} is the sum of all the aforementioned rewards. The state vector for CP includes I_1, I_2, t_{lock}, and V_{ctrl}, and a list of time stamps before the circuit is locked in. The state vector for VCDL is a list of frequencies at different V_{ctrl}.

3) Successive Approximation Register ADC: This experiment is to design a 10-bit 50MS/s asynchronous differential input SAR ADC with 28 tunable design parameters, including transistor sizes and passives. SAR ADC is a high-speed analog-to-digital converter that uses a binary search to gradually approach the digital value of the input analog signal, its circuit diagram is shown in Fig. 6. According to its circuit topology, it can be divided into four subblocks, i.e., bootstrap switch (BS), capacitive digital-to-analog converter (CDAC), comparator (COMP), and SAR logic circuit (SARL). BS completes the sampling and holding of the input voltage, its circuit diagram is shown in Fig. 6(b). CDAC converts the input signal of the SAR logic circuit into an analog voltage as an approximation value, as shown in Fig. 6(c). COMP compares the approximation value with the input signal and generates a digital bit according to the comparison result. There is also a peripheral function control circuit around COMP, as shown in Fig. 6(d). SARL in Fig. 6(e) approaches the value of the input signal by multiple incremental approximations, which are mainly composed of standard cells, so there is no device size task for SARL. Finally, we connect the output digital signal of SAR ADC to an ideal DAC and merge the redundant bits to simulate and analyze the converted analog signal.

The Spec targets of SAR ADC include (*a*) effective number of bits (ENOB), (*b*) signal-to-noise and distortion ratio (SNDR), (*c*) spurious-free dynamic range (SFDR), (*d*) total

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:45 UTC from IEEE Xplore. Restrictions apply.

harmonic distortion (THD), (*e*) power, and (*f*) FOM. The design objectives are defined as follows:

$$\begin{aligned}&\text{Effective Number of Bits} \geq 9.5 \text{ bit}\\&\text{Signal} - \text{to} - \text{Noise and Distortion Ratio} \geq 60 \text{ dB}\\&\text{Spurious} - \text{Free Dynamic Range} \geq 65 \text{ dB}\\&\text{Total Harmonic Distortion} \leq -60 \text{ dB}\\&\text{Power} \leq 8 \text{ mW}\\&\text{FOM} \leq 220\text{fJ/c.step.}\end{aligned} \tag{21}$$

In the SAR ADC circuit, a total of 28 independent design parameters are selected as optimization variables, including 14 design parameters in BS (W_{b1}, ... W_{b6}, L_{b1}, ... L_{b6}, C_{wb1}, and C_{sb1}), two design parameters in CDAC (C_{wc1} and C_{sc1}), and 12 design parameters in COMP (W_{com1}, ... W_{com6}, L_{com1}, ... L_{com6}), respectively. Based prior design experience, the search range of transistor width W is set to [2] and [16] μm, the search range of transistor length L is set to [0.06, 0.14] μm, the search range of capacitance width C_w and capacitance spacing C_s are set to [0.1, 0.16] μm.

A three agents RL framework is employed for this exercise, the agents of which are assigned to BS, CDAC, COMP and denoted as Agent^1, Agent^2, and Agent^3, respectively. The Specs of SAR ADC are normalized and added up as reward R_t as defined in (10). The boundary for ENOB is set between [0, 9.5] bit; the boundary for SNDR is set between [0, 60] dB; the boundary for SFDR is set between [0, 65] dB, the boundary for THD is set between [−60, 0] dB; the boundary for power of SAR ADC is set between [8], [10] mW, and r^t_{power} is set to −1 with power greater than 10 mW.

For Agent^1 corresponding to BS, ENOB, and SFDR are normalized using (9) as r^1_1 and r^1_2, respectively, and added as reward R^1. For Agent^2 corresponding to CDAC, area is used as reward and weighted sum as reward R^2. For agent 3 corresponding to COMP, delay time is used as reward R^3. The overall reward R_{tot} is the sum of all the aforementioned rewards. The state vector for Agent^1 and Agent^3 includes a list of V_{od} and the state vector for Agent^2 includes a list of voltage drops for each capacitor. Considering nonideal factors, we add a 1% mismatch bias to the capacitive components in the SAR ADC to simulate the effect of process bias, which better demonstrates the effectiveness of our approach in a real circuit environment.

B. Comparisons of MA-RL Under Different Reward and State Definitions

In the MA-RL framework, the *reward function* determines how agents evaluate different actions, while the *state* determines the circuit information that agents can observe at the moment. The definition of *reward function* and *state* vector may impact the optimization capability and robustness of the algorithm. In Section III-C, this article introduces two definitions for the state of agent i, State^{i,a} and State^{i,b}, and two definitions for the reward of the entire circuit $R_{t,a}$ and $R_{t,b}$. In this section, these different definitions of *rewards* and *states* in MA-RL and their impacts are investigated and discussed.

TABLE I
PERFORMANCE SPECS COMPARISONS OF GAIN BOOST AMPLIFIER OBTAINED BY TEN OPTIMIZATIONS OF MATD3 WITH TWO DIFFERENT REWARD SETTINGS

	Gain (dB)	UGF (MHz)	PM (°)	Power (mW)	SR (V/μs)	R_t
MATD3^{a1}	110.1	10.7	89.3	3.3	17.3	2.16
MATD3^{a2}	109.5	10.4	89.4	4.8	11.3	1.58
MATD3^{a3}	107.9	10.9	89.5	3.6	12.9	2.1
MATD3^{a4}	107.4	7.8	89.3	3.8	6.4	1.80
MATD3^{a5}	109.9	10.5	89.4	4.2	10.2	1.81
MATD3^{a6}	109.6	10.1	89.4	4.8	10.8	1.59
MATD3^{a7}	109.6	10.5	89.4	5.2	12.0	1.47
MATD3^{a8}	109.3	12.9	89.4	3.6	12.1	2.18
MATD3^{a9}	113.3	16.7	89.3	5.2	22.1	1.87
MATD3^{a10}	109.2	10.1	89.4	4.6	13.2	1.65
MATD3^{b1}	111.3	12.2	81.1	4.0	18.6	1.84
MATD3^{b2}	90.1	10.0	88.1	4.8	12.4	1.48
MATD3^{b3}	111.1	12.2	89.3	4.7	13.3	1.67
MATD3^{b4}	106.1	13.2	89.4	3.3	13.3	1.91
MATD3^{b5}	105.83	15.4	89.3	4.4	22.5	1.96
MATD3^{b6}	109.8	12.7	89.3	4.9	14.8	1.75
MATD3^{b7}	111.0	10.7	82.9	4.8	23.4	1.72
MATD3^{b8}	108.5	11.1	89.4	4.7	15.2	1.65
MATD3^{b9}	109.1	12.0	89.3	4.9	13.6	1.63
MATD3^{b10}	106.7	11.7	89.4	4.9	14	1.64

The effectiveness and robustness comparisons between MATD3 using $R_{t,a}$ and $R_{t,b}$ are performed, and the former is denoted as MATD3^a and the latter is denoted as MATD3^b, respectively. These two are both tested on the GBA circuit design for 10 runs, with each optimization run terminated after a fixed number of 15 000 simulations. The design objectives, ranges of design parameters, boundary values of Specs and weights of Specs for the overall GBA circuit and its subblock circuits are consistent with those set up for GBA in Section A. The results of this experiment are shown in Table I, where the Spec values marked in red indicate an unmet design Spec objective defined in (18). It is shown that MATD3^a fails for some of the design objectives in three out of ten runs, while *MATD3*^{b} fails only once. This shows that the reward with the clipping mechanism may help improve the robustness of the MA-RL methodology. Nevertheless, the single failure case of MATD3^{b2} indicates future work to improve other aspects of this framework is warranted to achieve a 100% success rate.

Fig. 7 depicts the results of ten separate optimization runs on the GBA circuit using the MATD3 algorithm under different agent *state* definitions. The circuits are evaluated using $R_{t,b}$ as a comprehensive assessment Spec. The red dashed line in the graph represents the baseline of $R_{t,b}$ that meets the design objective for the GBA ($R_{t,b} = 1.5$). MATD3^c shown in red in Fig. 7 uses the simulated operating conditions as the agent states [State^{i,a} in (5)]. The green data in Fig. 7 is obtained by MATD3^{d}, using the current performance of the corresponding circuit as the agent states [State^{i,b} in (6)]. It is shown that both MATD3^c and MATD3^d reach similar $R_{t,b}$ (1.95 for MATD3^c and 1.93 for MATD3^{d}). However, MATD3^c demonstrates a higher-success rate than that of MATD3^{d}. We suspect that the operating conditions of the circuit contain richer underlying information compared to the circuit Specs, and it can establish a more direct connection with the circuit design parameters

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:45 UTC from IEEE Xplore. Restrictions apply.

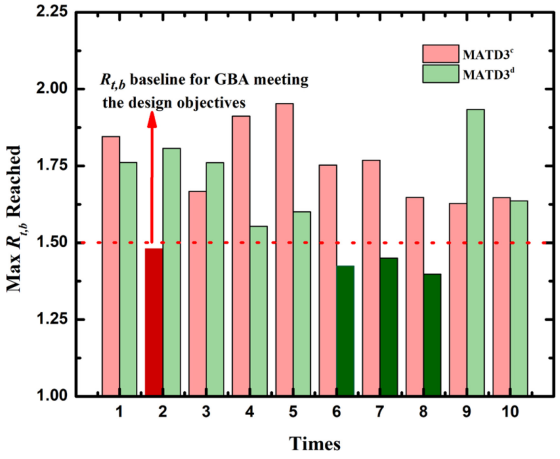


Fig. 7. Maximum $R_{t,b}$ reached after ten optimization runs of GBA using MATD3 with different state settings. MATD3^c uses the simulated operating conditions as the agent state vectors and MATD3^d uses the current performance of the corresponding circuit as the agent state vectors. The bold colors of red and green are failure cases for MATD3^c and MATD3^d, respectively.

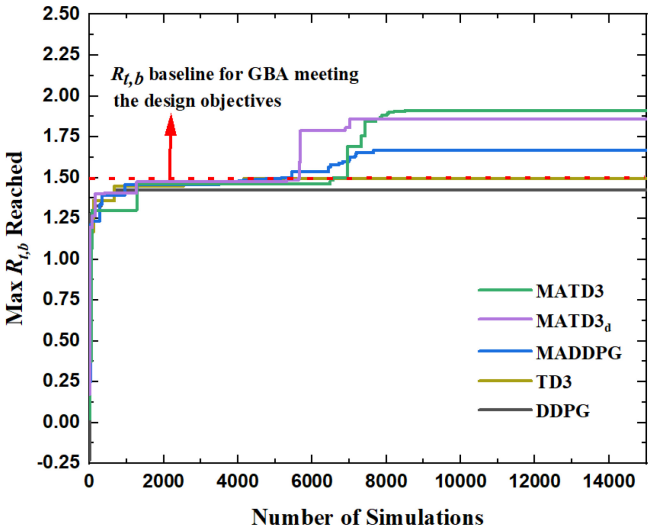


Fig. 8. Learning curves of five different RL algorithms using reward definition of $R_{t,b}$ tested on GBA design.

in the action network. Therefore, the strategy training of the actor network can be made more stable.

C. Results Comparisons Between Multiagent-Based RL and Other Circuit Optimization Methods

To demonstrate the effectiveness of the multiagent technique, a test using four RL algorithms on a GBA circuit is performed and the results are shown in Fig. 8. The circuits are evaluated using $R_{t,b}$ as a comprehensive assessment Spec. TD3 and MADDPG can be considered as two improved algorithms based on DDPG from different perspectives. The results from Fig. 8 indicate that both TD3 and multiagent technique are beneficial for reaching a more optimal result. Compared to the improvements from TD3, incorporating the multiagent structure indeed leads to a greater enhancement in the optimization performance from conventional DDPG.

TABLE II
PERFORMANCE METRICS COMPARISONS OF GBA

	Gain (dB)	UGF (MHz)	PM (°)	Power (mW)	SR (V/μs)	FoM
Human	107.3	6.5	88.5	3.9	8.8	5.0
Monte Carlo	101.1	6.6	89.7	5.3	4.2	4.0
BO	106.9	7.5	89.7	3.8	10.9	5.6
GA	106.8	7.4	89.7	3.8	9.7	5.4
DDPG	106.4	7.1	89.7	3.9	8.9	5.3
MADDPG	108.1	12.1	89.4	4.9	11.7	5.8
MAPPO	108.9	**12.2**	89.6	3.7	**17.9**	6.3
MATD3	**110.1**	10.7	89.3	**3.3**	17.3	**6.5**

Additionally, Fig. 8 also compares the optimization results on GBA using MATD3 with (MATD3$_d$) and without the "Done" flag (MATD3). The episode length for MATD3$_d$ is set to 40. As shown in Fig. 8, MATD3 achieves a more optimal result compared to MATD3$_d$, while MATD3$_d$ may converge faster. This may be attributed to the following reason. MATD3 tends to achieve a superior design even after meeting the targets within its episode length, and it does not terminate unless no higher reward is achieved for further 1000 consecutive simulations. Meanwhile, MATD3$_d$ resets the environment and reduces the expected future rewards upon meeting the design objectives, leading the agent to converge toward the termination condition of fulfilling the design targets.

Fig. 9(a)–(c) compares the learning curves of different algorithms for three different complex analog circuits, which are DLL, GBA, and ADC. The circuits are evaluated using the FoM in (2) as a comprehensive assessment Spec, and only the optimal design results are updated and displayed in learning curves during the simulation iteration process, i.e., the data point on the learning curves updates only if it is higher than the previous one. From Fig. 9, we see that all MA-RL methods outperform the conventional optimization algorithm, such as MC, GA, and BO, and they also achieve better results than the single agent RL DDPG on all three circuits. This indicates that the MA-RL methodology is indeed effective in the optimization tasks for complex analog circuits with multiple subblocks. Compared to single-agent RL methods (such as DDPG), MA-RL leverages a centralized learning and decentralized execution approach. This not only reduces the complexity of parameter optimization problems in complex analog circuits, but also balances the competition and collaboration among circuit subblocks. As a result, the MA-RL methods could reach more optimal design points in the complex among circuits.

Furthermore, for all test cases, the optimal design points optimized by MATD3 and MAPPO have higher FoMs than those optimized by MADDPG. Specifically, MATD3 performs optimally in GBA and SAR ADC designs, while MAPPO excels in optimizing DLL design. This suggests the effectiveness of multiagent twin-delayed techniques and multiagent PPO.

Finally, we also investigated each individual circuit specifications reached for all circuits, as shown in Tables II–IV. All MA-RL methods show good balance on all design specification targets in the three circuits. For GBA, our proposed

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:45 UTC from IEEE Xplore. Restrictions apply.

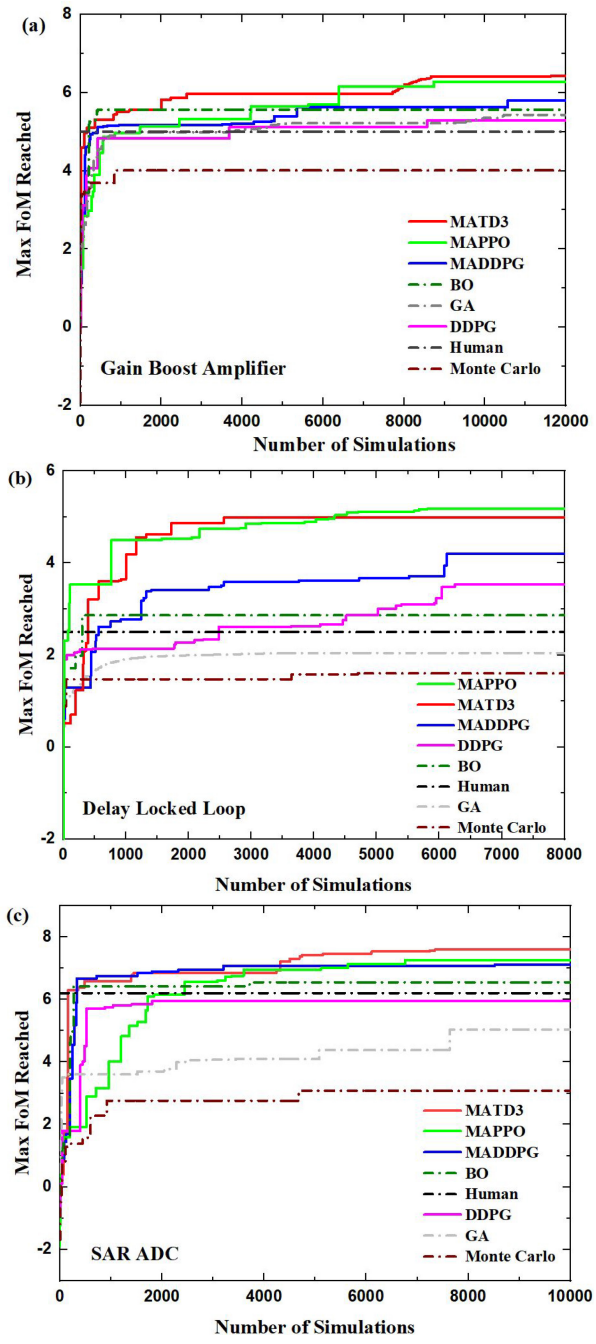


Fig. 9. Learning curves of different algorithms on (a) GBA design and (b) DLL design. (c) Successive approximation register ADC design. For all cases, the proposed MA-RL show higher FoMs, and the MATD3 and MAPPO generally achieve higher FoMs than that of MADDPG algorithm.

MATD3 obtains the best gain, minimum power with good UGF and reasonable PM, while MAPPO obtains the highest SR and the best UGF. For DLL, MATD3 achieves the lowest power, while MAPPO obtains the lowest-phase difference, widest working frequency range and the second best on lock time. For SAR ADC, MATD3 achieves the best ENOB, largest SNDR, largest FOM and second lowest THD, while MAPPO achieves the best SFDR and lowest power. All Tables demonstrate the high-quality design can be efficiently accomplished by our proposed methods. We have also done

TABLE III
PERFORMANCE SPECS COMPARISONS OF DLL

	PD (ps)	Lock time (us)	Working frequency range (MHz)	Power (mW)	Jitter (ps)	FoM
Human	82.6	0.52	150~300	18.6	191.0	2.5
Monte Carlo	103.2	0.47	170~280	43.6	442.0	1.6
BO	63.4	**0.35**	120~310	15.3	261.3	2.9
GA	76.3	0.43	116~305	22.5	119.8	2.0
DDPG	53.2	0.41	120~300	21.5	213.4	3.5
MADDPG	23.9	0.47	90~350	20.3	**180.7**	4.2
MATD3	17.5	0.38	70~370	**13.7**	193.9	4.9
MAPPO	**14.2**	0.36	**75~385**	17.1	193.2	**5.1**

TABLE IV
PERFORMANCE SPECS COMPARISONS OF SAR ADC

	ENOB (bit)	SNDR (dB)	SFDR (dB)	THD (dB)	Power (mW)	FOM (fJ/c.step)	FoM
Human	9.75	60.46	68.38	-67.99	6.77	157.2	6.2
Monte Carlo	9.21	57.21	63.56	-61.21	6.95	234.7	3.1
BO	9.81	60.83	68.94	-67.73	6.79	151.3	6.6
GA	9.55	59.26	67.93	-63.34	6.96	185.3	5.0
DDPG	9.69	60.12	69.65	-66.98	6.87	166.3	5.9
MADDPG	9.85	61.08	70.29	**-69.93**	6.81	147.6	7.1
MAPPO	9.84	61.02	**70.63**	-67.48	**6.72**	146.6	7.3
MATD3	**9.89**	**61.33**	70.07	-68.75	6.75	**142.3**	**7.6**

TABLE V
PERFORMANCE SPECS OF DLL FOR DIRECT MA-RL ACTION TRANSFER FROM 0.13 TO 0.18-MM TECHNOLOGY

	Phase difference (ps)	Lock time (us)	Working frequency range (MHz)	Power (mW)	Jitter (ps)
0.13μm	17.5	0.38	70~370	13.7	193.9
0.18μm	16.4	0.48	96-344	23.2	198.3

training speed comparisons under the same hardware (CPU) constraint, for a fixed number of simulation points, MA-RL methods are slightly (−5%) slower than single agent RL, while it is still more than 3X faster than BO.

D. Knowledge Transfer Performances

To show the optimality of the design created by our MA-RL methods, we apply the MATD3 trained action, i.e., the device sizes of DLL from 0.13-μm CMOS PDK directly to 0.18-μm PDK. The performance Specs of both are presented in Table V. It is demonstrated that the technology transfer is successful without any further change or optimization, except the V_{dd} is scaled from 1.2 to 1.8 V. Particularly, the phase difference and jitter are very close or even better than the original design, the lock time and WFR is also within design specification. Power is larger due to the V_{dd} scaling. This further shows the design point found by MA-RL method is of high quality.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:45 UTC from IEEE Xplore. Restrictions apply.

V. Conclusion

We propose the MA-RL-based methods for the automated optimization of complex analog circuits to achieve better performance. The circuit is proposed to be partitioned into subblocks based on topology information. Each subblock is assigned to an RL agent for which a specific reward is designed based on design knowledge. Then agents interact with each other during the training process to reach the overall design goal. Furthermore, the twin delayed technique and PPO are introduced to further improve the training stability and efficiency. Finally, adopting reward clipping and setting circuit operating conditions as agent states can help improve the success rate of the optimization algorithm in achieving design objectives. This work opens the pathway for large scale analog circuits as well as system design.

References

[1] I. A. M. Elfadel, D. S. Boning, and X. Li, *Machine Learning in VLSI Computer-Aided Design*. Cham, Switzerland: Springer, 2018.

[2] E. Deniz and G. Dündar, "Hierarchical performance estimation of analog blocks using Pareto Fronts," in *Proc. 6th Conf. Ph.D. Res. Microelectron. Electron.*, 2010, pp. 1–4.

[3] G. Alpaydin, S. Balkir, and G. Dundar, "An evolutionary approach to automatic synthesis of high-performance analog integrated circuits," *IEEE Trans. Evol. Comput.*, vol. 7, no. 3, pp. 240–252, Jun. 2003.

[4] M. Barros, J. Guilherme, and N. Horta, "Analog circuits optimization based on evolutionary computation techniques," *Integration*, vol.43, no. 1, pp. 136–155, 2010.

[5] G. Wolfe and R. Vemuri, "Extraction and use of neural network models in automated synthesis of operational amplifiers," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 22, no. 2, pp. 198–212, Feb. 2003.

[6] J. P. S. Rosa, D. J. D. Guerra, N. C. G. Horta, R. M. F. Martins, and N. C. C. Lourenço, *Using ANNs to Size Analog Integrated Circuits*, Cham, Switzerland: Springer Int. Publ., 2019.

[7] W. L. Lyu, F. Yang, C. H. Yan, D. A. Zhou, and X. Zeng, "Batch Bayesian optimization via multi-objective acquisition ensemble for automated analog circuit design," in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*, 2018, pp. 10–15.

[8] W. Lyu et al., "An efficient Bayesian optimization approach for automated optimization of analog circuits," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 6, pp. 1954–1967, Jun. 2018.

[9] S. Zhang, F. Yang, C. Yan, D. Zhou, and X. Zeng, "An efficient batch-constrained Bayesian optimization approach for analog circuit synthesis via multiobjective acquisition ensemble," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 1, pp. 1–14, Jan. 2022.

[10] H. Wang et al., "GCN-RL circuit designer: Transferable transistor sizing with graph neural networks and reinforcement learning," in *Proc. 57th ACM/IEEE Design Autom. Conf. (DAC)*, 2020, pp. 1–6.

[11] N. S. Karthik Somayaji, H. Hu, and P. Li, "Prioritized reinforcement learning for analog circuit optimization with design knowledge," in *Proc. 58th ACM/IEEE Design Autom. Conf. (DAC)*, 2021, pp. 1231–1236.

[12] K. Settaluri, A. Haj-Ali, Q. Huang, K. Hakhamaneshi, and B. Nikolic, "AutoCkt: Deep reinforcement learning of analog circuit designs," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, 2020, pp. 490–495.

[13] K. Settaluri, Z. Liu, R. Khurana, A. Mirhaj, R. Jain, and B. Nikolic, "Automated design of analog circuits using reinforcement learning," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 9, pp. 2794–2807, Sep. 2022.

[14] T. Sripramong and C. Toumazou, "The invention of CMOS amplifiers using genetic programming and current-flow analysis," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 21, no. 11, pp. 1237–1252, Nov. 2002.

[15] X. Wang and L. Hedrich, "An approach to topology synthesis of analog circuits using hierarchical blocks and symbolic analysis," in *Proc. Asia South Pac. Conf. Design Autom. (ASPDAC)*, 2006, pp. 700–705.

[16] H. Chen et al., "MAGICAL: An open- source fully automated analog IC layout system from netlist to GDSII," *IEEE Design Test*, vol. 38, no. 2, pp. 19–26, Apr. 2021.

[17] H. Habal and H. Graeb, "Constraint-based layout-driven sizing of analog circuits," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 30, no. 8, pp. 1089–1102, Aug. 2011.

[18] M. Fakhfakh, Y. Cooren, A. Sallem, M. Loulou, and P. Siarry, "Analog circuit design optimization through the particle swarm optimization technique," *Analog Integr. Circuits Signal Process.*, vol. 63, no. 1, pp. 71–82, 2011.

[19] M. del Mar Hershenson, S. S. Mohan, S. P. Boyd, and T. H. Lee, "Optimization of inductor circuits via geoSpec programming," in *Proc. Design Autom. Conf. (DAC)*, 1999, pp. 994–998.

[20] A. K. Singh, K. Ragab, M. Lok, C. Caramanis, and M. Orshansky, "Predictable equation-based analog optimization based on explicit capture of modeling error statistics," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 31, no. 10, pp. 1485–1498, Oct. 2012.

[21] Y. Li, Y. Wang, Y. Li, R. Zhou, and Z. Lin, "An artificial neural network assisted optimization system for analog design space exploration," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 10, pp. 2640–2653, Oct. 2020.

[22] B. Liu, D. Zhao, P. Reynaert, and G. G. E. Gielen, "GASPAD: A general and efficient mm-wave integrated circuit synthesis method based on surrogate model assisted evolutionary algorithm," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 33, no. 2, pp. 169–182, Feb. 2014.

[23] Z. Zhao and L. Zhang, "Deep reinforcement learning for analog circuit sizing," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2020, pp. 1–5.

[24] E. Ephrati and J. S. Rosenschein, "Divide and conquer in multi-agent planning," in *Proc. Int. Conf. Assoc. Adv. Artif. Intell.*, 1994, pp. 375–380.

[25] G. Berkol, E. Afacan, G. Dündar, and E. V. Fernandez, "A hierarchical design automation concept for analog circuits," in *Proc. IEEE Int. Conf. Electron., Circuits Syst. (ICECS)*, 2016, pp. 133–136.

[26] M. Neuner and H. Graeb, "Hierarchical analog power-down synthesis," in *Proc. 27th IEEE Int. Conf. Electronics, Circuits Syst. (ICECS)*, 2020, pp. 1–4.

[27] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[28] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," 2015, *arXiv:1509.02971*.

[29] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor–critic for mixed cooperative-competitive environments," in *Proc. 31st Annu. Conf. Neural Inf. Process. Syst. (NIPS)*, 2017, pp. 6382–6393.

[30] D. Silver, G. Lever, N. Heess, T. Degris, and M. Riedmiller, "Deterministic policy gradient algorithms," in *Proc. 31st Int. Conf. Mach. Learn. (ICML)*, 2014, pp. 387–395.

[31] S. Fujimoto, H. V. Hoof, and D. Meger, "Addressing function approximation error in actor–critic methods," in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*, 2018, pp. 1587–1596.

[32] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.

[33] J. Wang, L. Siek, R. Filippi, and K. A. Ng, "A top-down design verification based on reuse modular and paraspec behavioral modeling for subranging pipelined analog-to-digital converter," in *Proc. Int. Symp. Integr. Circuits (ISIC)*, 2007, pp. 378–381.

[34] W. Shi et al., "A robustanalog: Fast variation-aware analog circuit design via multi-task RL," in *Proc. ACM/IEEE 4th Workshop Mach. Learn. CAD (MLCAD)*, 2022, pp. 35–41.

[35] Z. Li and A. C. Carusone, "Design and optimization of low-dropout voltage regulator using relational graph neural network and reinforcement learning in open-source SKY130 process," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, 2023, pp. 1–9.



Jiarui Bao received the B.S. degree in material physics from Nanchang University, Nanchang, China, in 2018. He is currently pursuing the Ph.D. degree with the School of Information Science and Technology, Fudan University, Shanghai, China.

His research interests include analog circuit design automation and optimization, RF circuit design automation, and semiconductor devices and physics.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:45 UTC from IEEE Xplore. Restrictions apply.



Jinxin Zhang received the B.S. degree in information engineering from Shandong University, Jinan, Shandong, China, in 2020, and the M.S. degree from Fudan University, Shanghai, China, in 2023.

His research interests include analog circuit design automation and reinforcement learning.



Zhangcheng Huang (Member, IEEE) received the B.S. degree in physics from Nanjing University, Nanjing, China, in 2006, and the Ph.D. degree in microelectronics from the Chinese Academy of Sciences, Beijing, China, in 2011.

From 2011 to 2013, he was an Assistant Professor with the Shanghai Institute of Technical Physics, Chinese Academy of Sciences. From 2013 to 2020, he was an Associate Professor. He is currently an Associate Professor with the Frontier Institute of Chip and System, Fudan University, Shanghai, China. His research interests include high-performance ASICs for detectors and smart imaging sensors.



Zhaori Bi (Member, IEEE) received the B.Eng. degree in electronic information engineering from Wuhan University of Technology, Wuhan, China, in 2011, the M.S. and Ph.D. degrees in electrical engineering and computer engineering from The University of Texas at Dallas, Richardson, TX, USA, in 2013 and 2017, respectively.

He is currently an Assistant Professor with Fudan University, Shanghai, China. His current research interests include mixed-signal system-on-a-chip design, circuit performance optimization, and applications in medical AI.



Xingwei Feng received the B.S. degree in applied physics from Nanjing University of Science and Technology, Nanjing, China, in 2022. He is currently pursuing the master's degree in electronic information with Fudan University, Shanghai, China.

His research focuses on analog circuit design automation and reinforcement learning.



Xuan Zeng (Senior Member, IEEE) received the B.S. and Ph.D. degrees in electrical engineering from Fudan University, Shanghai, China, in 1991 and 1997, respectively.

She is currently a Full Professor with the Microelectronics Department. She served as the Director of the State Key Laboratory of Application Specific Integrated Circuits and Systems, Fudan University from 2008 to 2012. She was a Visiting Professor with the Department of Electrical Engineering, Texas A&M University, College Station, TX, USA, and with the Microelectronics Department, Technische Universiteit Delft, Delft, The Netherlands, in 2002 and 2003, respectively. Her current research interests include analog circuit modeling and synthesis, design for manufacturability, high-speed interconnect analysis and optimization, and circuit simulation.

Prof. Zeng received the Best Paper Award from *Integration, the VLSI Journal* 2018 and the Best Paper Award from the 8th IEEE Annual Ubiquitous Computing, Electronics and Mobile Communication Conference 2017. She received the Changjiang Distinguished Professor with the Ministry of Education Department of China in 2014, the Chinese National Science Funds for Distinguished Young Scientists in 2011 and the First-Class of Natural Science Prize of Shanghai in 2012, 10th For Women in Science Award in China in 2013, and Shanghai Municipal Natural Science Peony Award in 2014. She is an Associate Editor of IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS: PART II, IEEE TRANSACTIONS ON COMPUTER AIDED DESIGN OF INTEGRATED CIRCUITS AND SYSTEMS, and ACM TRANSACTIONS ON DESIGN AUTOMATION ON ELECTRONICS AND SYSTEMS.



Ye Lu (Member, IEEE) received the B.S. degree in physics from Nanjing University, Nanjing, China, in 2006, and the Ph.D. degree in physics from the University of Pennsylvania, Philadelphia, PA, USA, in 2011.

He was a Senior Research and Development Engineer with Intel Corporation, Hillsboro, OR, USA, from 2011 to 2016, and a Staff Engineer with Qualcomm, San Diego, CA, USA, from 2016 to 2019. He is currently a Faculty Member with Fudan University, Shanghai, China, where his research interests focus on semiconductor device, device modeling, automated circuit design, and simulations.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:45 UTC from IEEE Xplore. Restrictions apply.